

VentureMiner AI

Testing Strategy

Document: 08-Engineering / 22_testing

Version: v1.0

Date: 2026-07-20

Author: VentureMiner AI Documentation Team

Status: Approved

Project: VentureMiner AI — AI Venture Intelligence Platform

Table of Contents

Document 22 — Testing Strategy

Table of Contents

- 1. Purpose & Scope
- 2. Test pyramid
- 3. Unit testing
- 4. Integration testing
- 5. Contract testing
- 6. End-to-end testing
 - 6.1 API E2E
 - 6.2 UI E2E
- 7. Performance testing
- 8. Security testing
- 9. AI evaluation
- 10. Chaos testing
- 11. Test data
- 12. Test environments
- 13. Coverage
- 14. Test ownership
- 15. Appendix
 - 15.1 Revision history
 - 15.2 Cross-references

Document 22 — Testing Strategy

The contract for how we test the platform — unit, integration, contract, end-to-end, performance, security, AI evaluation, and chaos.

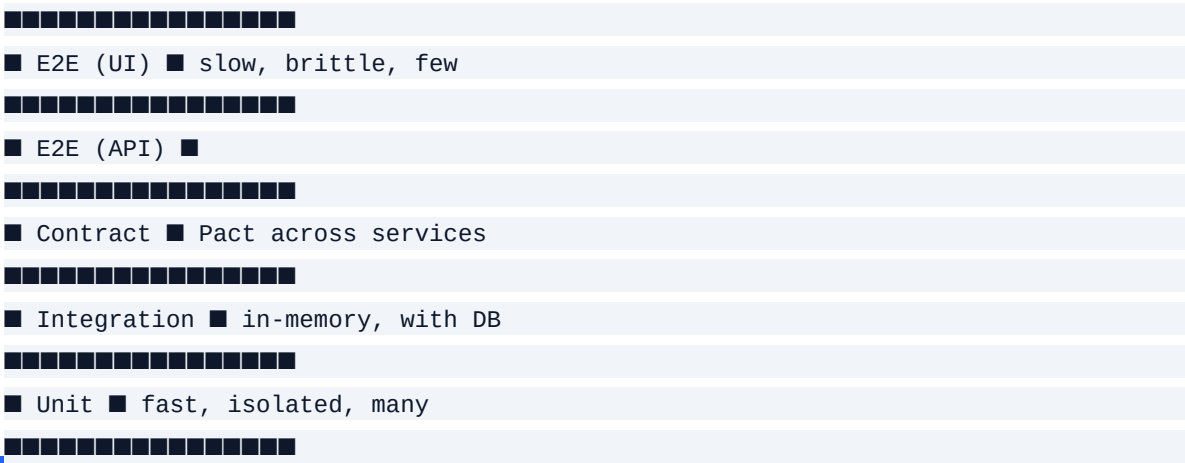
Table of Contents

- 1. Purpose & Scope
- 2. Test pyramid
- 3. Unit testing
- 4. Integration testing
- 5. Contract testing
- 6. End-to-end testing
- 7. Performance testing
- 8. Security testing
- 9. AI evaluation
- 10. Chaos testing
- 11. Test data
- 12. Test environments
- 13. Coverage
- 14. Test ownership
- 15. Appendix

1. Purpose & Scope

This document defines the testing strategy for the platform. It is the source of truth for which tests exist, at which layer, with which tools, and to what standard.

2. Test pyramid



- Coverage target: 85% per service.
- Tooling: pytest (Python), vitest (TypeScript).
- Style: AAA, no global state, no I/O.
- Mocks: at the boundary (DB, network, time).
- Property tests (Hypothesis) for parsers, validators, scoring math.

4. Integration testing

- Per service: spin up the service + its DB + its dependencies (via testcontainers).
- Scope: the service's public surface.
- Tooling: pytest + testcontainers-python, vitest + testcontainers-node.
- Coverage target: every endpoint, every error path, every event emitted.

5. Contract testing

- Tool: Pact.
- Scope: every cross-service boundary.
- Flow: consumer writes a contract; provider verifies.
- CI: contract tests run on every PR; broken contracts block merge.

6. End-to-end testing

6.1 API E2E

- Tool: Playwright (API), pytest for sync flows.
- Scope: the documented user journeys (Document 03).
- CI: nightly; full E2E on every release branch.

6.2 UI E2E

- Tool: Playwright.
- Scope: the 10 most critical user journeys.
- Visual regression: Percy.
- A11y: axe-core on every E2E.

7. Performance testing

- Tool: k6 (load), Locust (long-running).
- Scope: all public endpoints, the AI plane, the report pipeline.
- Cadence: weekly smoke; pre-release full.

8. Security testing

- SAST: Snyk in CI; daily full scan.
- DAST: OWASP ZAP weekly against staging.
- Dependency: Snyk + Dependabot; CVE alerts in tracker.
- Pen test: annual + on major release.
- Red team: annual for AI plane.

9. AI evaluation

- Golden sets: Document 18.
- Cadence: weekly (offline), continuous (online).
- Specific tests:
 - Citation precision and recall.
 - Verifier pass rate.
 - Score calibration MAE.
 - Tool-call success rate.
 - Output policy compliance.
 - Regressions > 2% block release.

10. Chaos testing

- Tool: AWS Fault Injection Service, plus custom Litmus scenarios.
- Scope:
 - DB primary failure.
 - Cache down.
 - LLM provider down.
 - NATS unavailable.
 - Worker crash.
- Cadence: monthly in staging; quarterly in production (controlled).

11. Test data

- Production-shaped, anonymized: cloned from prod for staging tests; PII redacted at the source.
- Synthetic: for new features and edge cases.
- Seed scripts: checked in for reproducibility.

12. Test environments

dev (local)	developer iteration	synthetic	local
CI	per-PR checks	synthetic	ephemeral
staging	integration + perf	anonymized	persistent
canary	production-shaped	production (mirrored)	persistent
production	live	customer	persistent

13. Coverage

- Unit: ≥ 85% per service.
- Integration: 100% of public surface.
- E2E: 10 critical journeys.
- AI eval: weekly golden set, regression on any dimension > 2%.

Coverage is reported in CI; drops are flagged but not blocking at the line level (we don't play coverage games).

14. Test ownership

- Every service owns its unit + integration + contract tests.
- A central QA team owns E2E, performance, security, AI evaluation.
- On-call owns chaos tests in production.

15. Appendix

15.1 Revision history

v0.5	2026-07-20	Doc Team	All sections drafted
v1.0	2026-07-20	Doc Team	First approved version

15.2 Cross-references

- TRD: Document 02.
- Backend: Document 05.
- AI Architecture: Documents 07–18.
- Operations: Document 28.

End of Document 22 — Testing Strategy. Testing is a habit, not a phase.